JavaScript లో OOP - పూర్తి తెలుగు గైడ్

> *ఈ document చదివిన తర్వాత OOP మళ్ళీ మర్చిపోలేవు. ప్రతి concept కి real-life scenario, వివరణ, మరియు code ఉంటాయి.*

విషయ సూచిక (Table of Contents)

1. OOP అంటే ఏమిటి?
2. Class మరియు Object
3. Constructor
4. this keyword
5. Encapsulation (Private Fields)
6. Getters మరియు Setters
7. Inheritance (వారసత్వం)
8. Constructor Chaining - super()
9. Method Overriding
10. Abstraction
11. Polymorphism (Runtime + Compile-time)
12. Static Methods మరియు Properties
13. Prototype Chain
14. Mixins - Multiple Inheritance Simulation
15. Diamond Problem
16. Method Chaining
17. Composition vs Inheritance
18. Complete Real-world Example (School System)
19. Memory Tips Table
20. Private Methods
21. Custom Error Classes (Error Inheritance)
22. toString() మరియు valueOf() Override
23. Object.create()
24. Factory Functions
25. Singleton Pattern
26. Object Immutability - Object.freeze() మరియు Object.seal()
27. Symbol.iterator - Objects ని Iterable చేయడం
28. Duck Typing
29. Protected Simulation (Public/Private/Protected)
30. Operator Overloading Simulation
31. bind(), call(), apply() - this ని Explicit గా Bind చేయడం
32. SOLID Principles
33. Modern Class Features (Static Blocks, Brand Checks, Symbol.hasInstance)
34. Async in OOP (async methods, Promises, for await)
35. Generators మరియు Iterators (function*, yield)
36. Class Fields మరియు Arrow Methods (Auto-bind)
37. Symbols, new.target మరియు Reflection

---

1. OOP అంటే ఏమిటి?

వివరణ

OOP అంటే Object-Oriented Programming - అంటే programming ని objects చుట్టూ నిర్మించడం.

Real-life Scenario

మీరు ఒక Car గురించి ఆలోచించండి:

- Car కి properties ఉంటాయి - model, color, year
- Car కి actions ఉంటాయి - start(), accelerate(), brake()

OOP లో, ఈ real-world ని code లో represent చేయడానికి Class మరియు Object వాడతాం.

OOP యొక్క 4 Pillars (స్తంభాలు)

Pillar	తెలుగు అర్థం	సంక్షిప్తం
Encapsulation	చుట్టుముట్టడం	Data ని దాచడం, safely access ఇవ్వడం
Abstraction	సంగ్రహణ	Implementation దాచి, interface మాత్రమే చూపించడం
Inheritance	వారసత్వం	ఒక class నుండి మరొక class properties తీసుకోవడం
Polymorphism	బహురూపం	ఒకే method వేర్వేరు రూపాల్లో behave చేయడం

---

2. Class మరియు Object

వివరణ

- **Class** అంటే ఒక blueprint (నమూనా). దాని నుండి objects create చేయవచ్చు.
- **Object** అంటే ఆ blueprint నుండి తయారు చేసిన actual item.

Real-life Scenario

> ***Class** = Car factory mold (అచ్చు)*
> *****Object** = ఆ మోల్డ్ నుండి తయారైన actual car*

ఒకే class నుండి వేలాది objects తయారు చేయవచ్చు - ప్రతి object దాని స్వంత data తో.

Code

```
// Class definition - ఇది blueprint
class Car {
  constructor(model, year) {
    this.model = model;
    this.year = year;
  }

  displayDetails() {
    console.log(`Car Model: ${this.model}`);
    console.log(`Car Year: ${this.year}`);
  }
}

// Object creation - blueprint నుండి actual car తయారు చేయడం
const car1 = new Car("Toyota", 2020);
```

```
const car2 = new Car("BMW", 2023);

car1.displayDetails(); // Car Model: Toyota, Car Year: 2020
car2.displayDetails(); // Car Model: BMW, Car Year: 2023
```

Key Points

- `class` keyword తో class define చేస్తాం
- `new` keyword తో object create చేస్తాం
- car1 మరియు car2 రెండూ వేర్వేరు objects - వేర్వేరు data కానీ same structure
- ఒక class నుండి unlimited objects తయారు చేయవచ్చు

---

3. Constructor

వివరణ

Constructor అంటే object create అయినప్పుడు **automatically** పిలవబడే special method.
దీని పని - object యొక్క initial values (properties) set చేయడం.

Real-life Scenario

> *Car factory లో car తయారు అయినప్పుడు, automatically color పెట్టడం, engine fit చేయడం జరుగుతుంది - అదే constructor చేసే పని.*

Code

```
class Student {
  // Constructor – object create అయినప్పుడు ఒక్కసారే నడుస్తుంది
  constructor(name, rollNo, grade) {
    this.name = name;
    this.rollNo = rollNo;
    this.grade = grade;
    console.log(`Student "${name}" created!`);
  }

  displayInfo() {
    console.log(
      `Name: ${this.name}, Roll: ${this.rollNo}, Grade: ${this.grade}`,
    );
  }
}

const s1 = new Student("Yaswanth", 101, "A"); // "Student Yaswanth created!" వస్తుంది
const s2 = new Student("Chaitanya", 102, "B");

s1.displayInfo(); // Name: Yaswanth, Roll: 101, Grade: A
s2.displayInfo(); // Name: Chaitanya, Roll: 102, Grade: B
```

Key Points

- Constructor పేరు ఎప్పుడూ `constructor` గా ఉంటుంది

- `new ClassName()` అని రాస్తే constructor automatically call అవుతుంది
- Constructor లో ఏ code రాసినా, object తయారైనప్పుడు execute అవుతుంది
- ఒక class లో ఒకే ఒక constructor ఉంటుంది (JavaScript లో)

Constructor లేకుండా జరిగే పని

```
class Dog {
  // Constructor రాయకపోతే, JavaScript automatically empty constructor add చేస్తుంది
  // అంటే: constructor() {} — ఇది automatically ఉంటుంది
  bark() {
    console.log("Woof!");
  }
}

const dog = new Dog(); // works fine, empty constructor వాడతుంది
dog.bark(); // Woof!
```

---

4. this keyword

వివరణ

`this` అంటే **current object** ని refer చేస్తుంది.
Method లో `this` వాడినప్పుడు, ఆ method ని ఏ object పిలిచిందో ఆ object ని చూపిస్తుంది.

Real-life Scenario

> *మీరు "నా పేరు Yaswanth" అంటే - "నా" అనే word మీ గురించే మాట్లాడుతోంది.*
> *అదే విధంగా code లో `this` ఆ specific object గురించి మాట్లాడుతుంది.*

Code

```
class Person {
  constructor(name, age) {
    this.name = name; // "this" = ఇప్పుడు create అవుతున్న object
    this.age = age;
  }

  greet() {
    console.log(`హాలో! నా పేరు ${this.name}, నా వయసు ${this.age}`);
  }
}

const p1 = new Person("Yaswanth", 22);
const p2 = new Person("Chaitanya", 25);

p1.greet(); // హాలో! నా పేరు Yaswanth, నా వయసు 22
p2.greet(); // హాలో! నా పేరు Chaitanya, నా వయసు 25
```

```
// p1.greet() లో this = p1 (p1 object)
// p2.greet() లో this = p2 (p2 object)
```

this యొక్క Gotcha - Arrow Functions vs Regular Functions

```
class Timer {
  constructor() {
    this.count = 0;
  }

  // WRONG: regular function లో this setTimeout callback లో మారిపోతుంది
  startWrong() {
    setTimeout(function () {
      // Sloppy mode: this = global object (browser లో window) → this.count =
undefined
      // Strict mode / ES modules: this = undefined → ఈ line TypeError throw చేస్తుంది
      this.count++;
      console.log(this.count); // Timer object కాదు! (sloppy mode లో NaN)
    }, 1000);
  }

  // CORRECT: arrow function తో this correctly works
  startCorrect() {
    setTimeout(() => {
      this.count++; // ఇక్కడ this = Timer object — CORRECT!
      console.log(this.count); // 1
    }, 1000);
  }
}

const t = new Timer();
t.startCorrect();
```

> ***Rule:*** *Arrow functions లో `this` parent scope నుండి తీసుకుంటుంది.*
> *Regular functions లో `this` caller బట్టి మారుతుంది.*

---

5. Encapsulation

వివరణ

Encapsulation అంటే data ని బయటివారి నుండి దాచడం మరియు controlled access మాత్రమే ఇవ్వడం.

Real-life Scenario

> ***ATM machine*** *లో మీరు balance చూడవచ్చు, డబ్బు తీసుకోవచ్చు - కానీ లోపలి circuits, database direct గా touch చేయలేరు.*
> *ATM అనే machine data ని encapsulate చేస్తుంది. Access కోసం PIN, buttons ఉన్నాయి (controlled access).*

Code - Private Fields తో (`#` symbol)

```javascript
class BankAccount {
  #balance; // Private field - బయటి నుండి directly access చేయలేరు
  #accountNumber; // Private field

  constructor(accountNumber, initialBalance) {
    this.#accountNumber = accountNumber;
    this.#balance = initialBalance;
    this.owner = "Yaswanth"; // Public - anyone can access
  }

  // Controlled access - method ద్వారా మాత్రమే
  deposit(amount) {
    if (amount <= 0) {
      console.log("Invalid deposit amount!");
      return;
    }
    this.#balance += amount;
    console.log(
      `Rs.${amount} deposit చేయబడింది. Current balance: Rs.${this.#balance}`,
    );
  }

  withdraw(amount) {
    if (amount > this.#balance) {
      console.log("Insufficient balance!");
      return;
    }
    this.#balance -= amount;
    console.log(
      `Rs.${amount} withdraw చేయబడింది. Remaining: Rs.${this.#balance}`,
    );
  }

  getBalance() {
    return this.#balance; // Read-only access
  }

  getAccountNumber() {
    // Security కోసం - last 4 digits మాత్రమే చూపించడం
    return "****" + this.#accountNumber.toString().slice(-4);
  }
}

const account = new BankAccount(1234567890, 5000);

// Allowed actions
account.deposit(1000); // Rs.1000 deposit చేయబడింది. Current balance: Rs.6000
account.withdraw(2000); // Rs.2000 withdraw చేయబడింది. Remaining: Rs.4000
console.log(account.getBalance()); // 4000
console.log(account.getAccountNumber()); // ****7890
```

```
// Direct access కుదరదు
// console.log(account.#balance); // SyntaxError: Private field '#balance'
// account.#balance = 99999;       // SyntaxError - hack చేయలేరు!
```

Key Points

- `#` తో మొదలయ్యే properties **private** - class బయట access చేయలేరు
- Data protection కోసం encapsulation వాడతాం
- Validation తో controlled గా set/get చేస్తాం
- Public methods (deposit, withdraw) ద్వారా మాత్రమే private data access అవుతుంది

---

6. Getters మరియు Setters

వివరణ

Getter - property చదివేటప్పుడు automatically call అయ్యే method.
Setter - property set చేసేటప్పుడు automatically call అయ్యే method.

Real-life Scenario

> *Temperature display చేసే thermometer:*
>
> - *Celsius లో store చేస్తాం*
> - *Fahrenheit లో display చేసేటప్పుడు getter automatically convert చేస్తుంది*

Code

```
class Temperature {
  #celsius;

  constructor(celsius) {
    this.#celsius = celsius;
  }

  // Getter - property లా access చేస్తాం కానీ method లా పని చేస్తుంది
  get fahrenheit() {
    return (this.#celsius * 9) / 5 + 32;
  }

  get celsius() {
    return this.#celsius;
  }

  // Setter - validation తో set చేయడం
  set celsius(value) {
    if (value < -273.15) {
      console.log("Absolute zero కంటే తక్కువ కాదు!");
      return;
    }
    this.#celsius = value;
  }
```

```
}

const temp = new Temperature(100);

// Getter వాడటం - method call లా కాదు, property లా వాడతాం
console.log(temp.celsius); // 100  (getter call అవుతుంది)
console.log(temp.fahrenheit); // 212  (getter automatically convert చేస్తుంది)

// Setter వాడటం - assignment తో automatic validation జరుగుతుంది
temp.celsius = 37;
console.log(temp.celsius); // 37

temp.celsius = -300; // Absolute zero కంటే తక్కువ కాదు!
console.log(temp.celsius); // 37 (unchanged - validation block చేసింది)
```

User Profile Example

```
class User {
  #firstName;
  #lastName;

  constructor(firstName, lastName) {
    this.#firstName = firstName;
    this.#lastName = lastName;
  }

  // Computed property getter
  get fullName() {
    return `${this.#firstName} ${this.#lastName}`;
  }

  set fullName(name) {
    const parts = name.split(" ");
    this.#firstName = parts[0];
    this.#lastName = parts[1] || "";
  }

  get firstName() {
    return this.#firstName;
  }
  get lastName() {
    return this.#lastName;
  }
}

const user = new User("Yaswanth", "Bonumaddi");
console.log(user.fullName); // Yaswanth Bonumaddi

user.fullName = "Chaitanya Kumar";
```

```
console.log(user.fullName); // Chaitanya Kumar
console.log(user.firstName); // Chaitanya
```

---

7. Inheritance (వారసత్వం)

వివరణ

Inheritance అంటే ఒక class (parent/base) యొక్క properties మరియు methods ని మరొక class (child/derived) తీసుకోవడం.
`extends` keyword వాడతాం.

Real-life Scenario

> ***Vehicle -> Car -> ElectricCar*** *అనే hierarchy చూడండి:*
>
> - *Vehicle: basic properties (wheels, engine)*
> - *Car: Vehicle అన్నీ + extra (doors, AC)*
> - *ElectricCar: Car అన్నీ + extra (battery, charging)*

Code

```
// Parent Class (Base Class)
class Vehicle {
  constructor(brand, speed) {
    this.brand = brand;
    this.speed = speed;
  }

  move() {
    console.log(`${this.brand} ${this.speed} km/h వేగంతో వెళ్తోంది.`);
  }

  stop() {
    console.log(`${this.brand} ఆగింది.`);
  }
}

// Child Class - Vehicle నుండి inherit చేయడం
class Car extends Vehicle {
  constructor(brand, speed, doors) {
    super(brand, speed); // Parent constructor call చేయడం (mandatory!)
    this.doors = doors;
  }

  honk() {
    console.log(`${this.brand} బీప్ బీప్!`);
  }
}

// Grandchild Class
```

```
class ElectricCar extends Car {
  constructor(brand, speed, doors, batteryRange) {
    super(brand, speed, doors); // Car constructor call చేయడం
    this.batteryRange = batteryRange;
  }

  charge() {
    console.log(`${this.brand} charging అవుతోంది...`);
  }
  // Car మరియు Vehicle యొక్క methods కూడా వాడవచ్చు!
}

// Objects తయారు చేయడం
const myVehicle = new Vehicle("Generic", 60);
const myCar = new Car("Toyota", 120, 4);
const myTesla = new ElectricCar("Tesla", 200, 4, 500);

myVehicle.move(); // Generic 60 km/h వేగంతో వెళ్తోంది.

myCar.move(); // Toyota 120 km/h వేగంతో వెళ్తోంది. (parent method వాడుతోంది)
myCar.honk(); // Toyota బీప్ బీప్!

myTesla.move(); // Tesla 200 km/h వేగంతో వెళ్తోంది. (grandparent method!)
myTesla.honk(); // Tesla బీప్ బీప్! (parent method!)
myTesla.charge(); // Tesla charging అవుతోంది...

// instanceof తో check చేయవచ్చు
console.log(myTesla instanceof ElectricCar); // true
console.log(myTesla instanceof Car); // true
console.log(myTesla instanceof Vehicle); // true – inheritance chain!
```

Inheritance Chain (వారసత్వ శ్రేణి)

```
Vehicle
  └── Car
        └── ElectricCar
```

ElectricCar object కి:

- ElectricCar methods (charge) తో సహా
- Car methods (honk) - inherited
- Vehicle methods (move, stop) - inherited from grandparent

---

8. Constructor Chaining - super()

వివరణ

Child class constructor లో `super()` తప్పనిసరిగా పిలవాలి - ఇది parent class constructor ని call చేస్తుంది. `super()` call చేయకముందు `this` వాడకూడదు - error వస్తుంది.

Real-life Scenario

> *Employee అనే class hire చేసేటప్పుడు, ముందు Person గా register అవ్వాలి (name, age), తర్వాత Employee details (department, salary) add అవుతాయి.*
> *అంటే - parent constructor ముందు run అవుతుంది, తర్వాత child constructor run అవుతుంది.*

Code

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
    console.log(`1 – Person constructor: ${name}`);
  }
}

class Employee extends Person {
  constructor(name, age, department, salary) {
    super(name, age); // ముందు Person constructor పిలవాలి
    // super() తర్వాత మాత్రమే this వాడవచ్చు
    this.department = department;
    this.salary = salary;
    console.log(`2 – Employee constructor: ${department}`);
  }

  displayInfo() {
    console.log(`Name: ${this.name}, Age: ${this.age}`);
    console.log(`Department: ${this.department}, Salary: Rs.${this.salary}`);
  }
}

class Manager extends Employee {
  constructor(name, age, department, salary, teamSize) {
    super(name, age, department, salary); // Employee constructor పిలవాలి
    this.teamSize = teamSize;
    console.log(`3 – Manager constructor: team of ${teamSize}`);
  }

  displayInfo() {
    super.displayInfo(); // Parent method call చేయడం
    console.log(`Team Size: ${this.teamSize}`);
  }
}

console.log("--- Manager object create అవుతోంది ---");
const mgr = new Manager("Yaswanth", 30, "Engineering", 150000, 10);

// Output order:
// 1 – Person constructor: Yaswanth     (first!)
// 2 – Employee constructor: Engineering
// 3 – Manager constructor: team of 10  (last!)

console.log("--- Display Info ---");
```

```
mgr.displayInfo();
// Name: Yaswanth, Age: 30
// Department: Engineering, Salary: Rs.150000
// Team Size: 10
```

super() లేకుండా error

```
class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    // super(name); // ఇది లేకుండా...
    this.breed = breed; // ReferenceError: Must call super constructor before this
  }
}
```

---

9. Method Overriding

వివరణ

Method Overriding అంటే child class లో parent class యొక్క method ని **same name తో** redefine చేయడం. Child class తన version తో parent version ని override చేస్తుంది.

Real-life Scenario

> *Animal sound() method ఉంది. Dog, Cat, Cow - అన్నీ animal కానీ sound() వేర్వేరుగా చేస్తాయి.*

Code

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  sound() {
    console.log(`${this.name} ఏదో sound చేస్తోంది.`);
  }

  sleep() {
    console.log(`${this.name} నిద్రపోతోంది.`);
  }
}

class Dog extends Animal {
  sound() {
```

```
    // Parent method override చేయడం
    console.log(`${this.name} భౌ భౌ అంటోంది!`);
  }
}

class Cat extends Animal {
  sound() {
    console.log(`${this.name} మ్యావ్ మ్యావ్ అంటోంది!`);
  }
}

class Cow extends Animal {
  sound() {
    console.log(`${this.name} అంబా అంటోంది!`);
  }
}

const dog = new Dog("Tommy");
const cat = new Cat("Kitty");
const cow = new Cow("Lakshmi");

dog.sound(); // Tommy భౌ భౌ అంటోంది!
cat.sound(); // Kitty మ్యావ్ మ్యావ్ అంటోంది!
cow.sound(); // Lakshmi అంబా అంటోంది!

// sleep() override చేయలేదు కాబట్టి parent version వాడతారు
dog.sleep(); // Tommy నిద్రపోతోంది.
```

super తో parent method కూడా call చేయడం

```
class Shape {
  draw() {
    console.log("Shape drawing...");
  }
}

class Circle extends Shape {
  draw() {
    super.draw(); // Parent method కూడా call చేయడం
    console.log("Circle drawing..."); // తర్వాత extra పని చేయడం
  }
}

const c = new Circle();
c.draw();
// Shape drawing...
// Circle drawing...
```

---

10. Abstraction

వివరణ

Abstraction అంటే లోపలి complex implementation దాచి, user కి only necessary interface చూపించడం.
JavaScript లో abstract classes officially లేవు - కానీ simulate చేయవచ్చు.

Real-life Scenario

> *మీరు **TV remote** వాడతారు - volume up button press చేస్తారు.*
> *కానీ లోపల IR signal ఎలా పనిచేస్తుంది, TV circuit ఎలా respond చేస్తుంది - మీకు తెలియదు మరియు తెలియవలసిన అవసరం లేదు.*
> *అదే abstraction - complexity దాచి simple interface ఇవ్వడం.*

Code

```
// Abstract Base Class simulation
class DatabaseConnection {
  constructor(host, port) {
    // Abstract class ని directly instantiate చేయకుండా నిరోధించడం
    if (new.target === DatabaseConnection) {
      throw new Error(
        "DatabaseConnection abstract class - directly instantiate చేయలేరు!",
      );
    }
    this.host = host;
    this.port = port;
  }

  // Abstract method simulation - subclass తప్పనిసరిగా implement చేయాలి
  connect() {
    throw new Error("connect() method implement చేయాలి!");
  }

  disconnect() {
    throw new Error("disconnect() method implement చేయాలి!");
  }

  // Concrete method - common logic అందరికీ same
  ping() {
    console.log(`${this.host}:${this.port} ని ping చేస్తున్నాం...`);
    this.connect(); // Template method pattern
  }
}

// Concrete class - abstract methods implement చేయడం
class MySQLConnection extends DatabaseConnection {
  connect() {
    console.log(`MySQL: ${this.host}:${this.port} కి connect అయింది!`);
  }

  disconnect() {
    console.log("MySQL connection closed.");
```

```
  }
}

class MongoDBConnection extends DatabaseConnection {
  connect() {
    console.log(`MongoDB: ${this.host}:${this.port} కి connect అయింది!`);
  }

  disconnect() {
    console.log("MongoDB connection closed.");
  }
}

// Abstract class directly instantiate చేయడానికి ప్రయత్నిస్తే
// const db = new DatabaseConnection("localhost", 3306); // Error!

// Concrete classes వాడవచ్చు
const mysql = new MySQLConnection("localhost", 3306);
mysql.ping(); // ping చేస్తున్నాం... -> MySQL connect అయింది!
mysql.disconnect(); // MySQL connection closed.

const mongo = new MongoDBConnection("cloud.db.com", 27017);
mongo.ping(); // MongoDB connect అయింది!
```

Abstraction యొక్క Benefits

1. **Complexity hidden** - user simple interface మాత్రమే చూస్తాడు
2. **Easy to change** - implementation మార్చినా user code affect కాదు
3. **Contract enforced** - subclass తప్పనిసరిగా methods implement చేయాలి

---

11. Polymorphism

వివరణ

Polymorphism = "Poly" (చాలా) + "morph" (రూపాలు)
అంటే same interface/method name కానీ వేర్వేరు classes లో వేర్వేరుగా behave చేయడం.

Runtime Polymorphism

Real-life Scenario

> *Payment system:*
>
> - *CreditCard.pay() -> credit card logic*
> - *UPI.pay() -> UPI logic*
> - *Cash.pay() -> cash logic*
>   *అన్నీ pay() method - కానీ behaviour వేర్వేరు!*

Code

```javascript
class Payment {
  pay(amount) {
    throw new Error("pay() implement చేయాలి!");
  }
}

class CreditCard extends Payment {
  constructor(cardNumber) {
    super();
    this.cardNumber = cardNumber;
  }

  pay(amount) {
    console.log(
      `Credit Card (****${this.cardNumber.slice(-4)}): Rs.${amount} payment
చేయబడింది!`,
    );
  }
}

class UPI extends Payment {
  constructor(upiId) {
    super();
    this.upiId = upiId;
  }

  pay(amount) {
    console.log(`UPI (${this.upiId}): Rs.${amount} transfer చేయబడింది!`);
  }
}

class Cash extends Payment {
  pay(amount) {
    console.log(`Cash: Rs.${amount} నగదు చెల్లించబడింది!`);
  }
}

// Polymorphism in action!
// అన్నీ Payment type - కానీ pay() వేర్వేరుగా behave చేస్తుంది
const payments = [
  new CreditCard("1234567890123456"),
  new UPI("yaswanth@upi"),
  new Cash(),
];

// ఒకే loop లో అన్నీ process చేయవచ్చు!
payments.forEach((payment) => {
  payment.pay(500); // Runtime లో which pay() call అవుతుందో decide అవుతుంది
});

// Output:
```

```
// Credit Card (****3456): Rs.500 payment చేయబడింది!
// UPI (yaswanth@upi): Rs.500 transfer చేయబడింది!
// Cash: Rs.500 నగదు చెల్లించబడింది!
```

Compile-time Polymorphism (Method Overloading Simulation)

వివరణ

Java లో **Method Overloading** ఉంది - same method name, different parameters.
JavaScript లో officially లేదు, కానీ rest parameters తో simulate చేయవచ్చు.

Code

```
class Calculator {
  // One method, multiple behaviors based on argument count
  add(...args) {
    if (args.length === 0) {
      return 0;
    } else if (args.length === 2) {
      console.log(`2 numbers: ${args[0]} + ${args[1]} = ${args[0] + args[1]}`);
      return args[0] + args[1];
    } else if (args.length === 3) {
      const sum = args[0] + args[1] + args[2];
      console.log(`3 numbers: ${args[0]} + ${args[1]} + ${args[2]} = ${sum}`);
      return sum;
    } else {
      const sum = args.reduce((acc, val) => acc + val, 0);
      console.log(`${args.length} numbers sum: ${sum}`);
      return sum;
    }
  }

  // Type based overloading simulation
  describe(value) {
    if (typeof value === "string") {
      console.log(`String: "${value}" - length: ${value.length}`);
    } else if (typeof value === "number") {
      console.log(`Number: ${value} - squared: ${value * value}`);
    } else if (Array.isArray(value)) {
      console.log(`Array: [${value}] - length: ${value.length}`);
    }
  }
}

const calc = new Calculator();
calc.add(10, 20); // 2 numbers: 10 + 20 = 30
calc.add(10, 20, 30); // 3 numbers: 10 + 20 + 30 = 60
calc.add(1, 2, 3, 4, 5); // 5 numbers sum: 15

calc.describe("Hello"); // String: "Hello" - length: 5
```

```
calc.describe(7); // Number: 7 - squared: 49
calc.describe([1, 2, 3]); // Array: [1,2,3] - length: 3
```

---

12. Static Methods మరియు Properties

వివరణ

Static అంటే class కి చెందినవి - object కి కాదు.
`ClassName.method()` తో call చేస్తాం - object create చేయవలసిన అవసరం లేదు.

Real-life Scenario

> ***Math.sqrt(16)*** *- Math object create చేయకుండా directly Math class method వాడతాం. అదే static!*

Code

```
class MathHelper {
  // Static property
  static PI = 3.14159265358979;

  // Static methods - object లేకుండా call చేయవచ్చు
  static square(n) {
    return n * n;
  }

  static cube(n) {
    return n * n * n;
  }

  static circleArea(radius) {
    return MathHelper.PI * MathHelper.square(radius);
  }

  static isPrime(n) {
    if (n < 2) return false;
    for (let i = 2; i <= Math.sqrt(n); i++) {
      if (n % i === 0) return false;
    }
    return true;
  }
}

// Object create చేయకుండా directly use చేయడం
console.log(MathHelper.PI); // 3.14159265358979
console.log(MathHelper.square(5)); // 25
console.log(MathHelper.cube(3)); // 27
console.log(MathHelper.circleArea(7)); // 153.93...
console.log(MathHelper.isPrime(17)); // true
console.log(MathHelper.isPrime(18)); // false
```

```
// Object తో static method call కుదరదు
// const m = new MathHelper();
// m.square(5); // TypeError: m.square is not a function
```

Counter Example - Static property shared across instances

```
class Counter {
  static count = 0; // Shared across all instances

  constructor(name) {
    this.name = name;
    Counter.count++; // Every object create అయినప్పుడు increment
    console.log(`${name} created. Total objects: ${Counter.count}`);
  }

  static getTotal() {
    return Counter.count;
  }
}

const c1 = new Counter("First"); // First created. Total objects: 1
const c2 = new Counter("Second"); // Second created. Total objects: 2
const c3 = new Counter("Third"); // Third created. Total objects: 3

console.log(Counter.getTotal()); // 3
```

---

13. Prototype Chain

వివరణ

JavaScript లో **Prototype** అనేది inheritance యొక్క foundation (పునాది).
ప్రతి object కి `__proto__` అనే hidden link ఉంటుంది - parent object ని point చేస్తుంది.

Method వెతికేటప్పుడు:

1. Object లో వెతుకుతుంది
2. లేకుంటే prototype లో వెతుకుతుంది
3. లేకుంటే prototype యొక్క prototype లో వెతుకుతుంది
4. null వచ్చే వరకు continue అవుతుంది

Real-life Scenario

> *మీరు ఒక word చదవలేకపోతే dictionary చూస్తారు, dictionary లో లేకుంటే encyclopedia చూస్తారు...*
> *అదే prototype chain!*

Code

```
class Animal {
  constructor(name) {
    this.name = name;
```

```
  }

  eat() {
    console.log(`${this.name} తింటోంది.`);
  }
}

class Dog extends Animal {
  bark() {
    console.log(`${this.name} భౌ అంటోంది!`);
  }
}

const dog = new Dog("Tommy");

// Prototype chain:
// dog -> Dog.prototype -> Animal.prototype -> Object.prototype -> null

dog.bark(); // Found in Dog.prototype
dog.eat(); // Not in Dog.prototype -> found in Animal.prototype
dog.toString(); // Not in Dog/Animal -> found in Object.prototype

// Prototype chain చూడడం
console.log(Object.getPrototypeOf(dog) === Dog.prototype); // true
console.log(Object.getPrototypeOf(Dog.prototype) === Animal.prototype); // true

// hasOwnProperty - own property vs inherited property
console.log(dog.hasOwnProperty("name")); // true (dog's own property)
console.log(dog.hasOwnProperty("bark")); // false (prototype లో ఉంది)
```

Visual Prototype Chain

```
dog object
  { name: "Tommy" }
      |
      | __proto__
      v
Dog.prototype
  { bark: [Function], constructor: Dog }
      |
      | __proto__
      v
Animal.prototype
  { eat: [Function], constructor: Animal }
      |
      | __proto__
      v
Object.prototype
  { toString, hasOwnProperty, valueOf, ... }
      |
      | __proto__
```

```
    v
null  <-- chain ends here
```

---

14. Mixins - Multiple Inheritance Simulation

వివరణ

JavaScript లో **Multiple Inheritance** లేదు (ఒక class ఒకే class extend చేయగలదు).
Mixins తో multiple sources నుండి behaviour compose చేయవచ్చు.

Real-life Scenario

> *ఒక **SmartPhone** అనేది:*
>
> - *Phone (call చేయడం)*
> - *Camera (photos తీయడం)*
> - *MusicPlayer (music వినడం)*
> *అన్నీ ఒకే device లో - కానీ Phone extends Camera, MusicPlayer కాదు.*
> *అదే mixins - unrelated behaviours combine చేయడం.*

Code

```javascript
// Mixin functions - reusable behaviours
const Flyable = (Base) =>
  class extends Base {
    fly() {
      console.log(`${this.name} ఎగురుతోంది!`);
    }

    land() {
      console.log(`${this.name} దిగింది.`);
    }
  };

const Swimmable = (Base) =>
  class extends Base {
    swim() {
      console.log(`${this.name} ఈదుతోంది!`);
    }
  };

const Runnable = (Base) =>
  class extends Base {
    run() {
      console.log(`${this.name} పరిగెత్తుతోంది!`);
    }
  };

// Base class
class Animal {
  constructor(name) {
```

```
    this.name = name;
  }
}

// Duck - fly + swim + run అన్నీ చేయగలదు
class Duck extends Flyable(Swimmable(Runnable(Animal))) {
  quack() {
    console.log(`${this.name} క్వాక్ క్వాక్!`);
  }
}

// Eagle - only fly చేయగలదు
class Eagle extends Flyable(Animal) {
  screech() {
    console.log(`${this.name} అరుస్తోంది!`);
  }
}

// Fish - only swim చేయగలదు
class Fish extends Swimmable(Animal) {
  blowBubbles() {
    console.log(`${this.name} bubbles వదులుతోంది!`);
  }
}

const duck = new Duck("Donald");
duck.fly(); // Donald ఎగురుతోంది!
duck.swim(); // Donald ఈదుతోంది!
duck.run(); // Donald పరిగెత్తుతోంది!
duck.quack(); // Donald క్వాక్ క్వాక్!

const eagle = new Eagle("Sam");
eagle.fly(); // Sam ఎగురుతోంది!
// eagle.swim(); // Error - Eagle swim చేయలేదు!

const fish = new Fish("Nemo");
fish.swim(); // Nemo ఈదుతోంది!
fish.blowBubbles(); // Nemo bubbles వదులుతోంది!
```

---

15. Diamond Problem

వివరణ

Diamond Problem అంటే - ఒక class రెండు classes extend చేసినప్పుడు మరియు ఆ రెండు classes లో same method ఉన్నప్పుడు అయ్యే ambiguity (అనిశ్చయత).

Real-life Scenario

> *HybridCar:*
>
> - *ElectricCar నుండి fuelType() -> "electricity"*

> - *PetrolCar నుండి fuelType() -> "petrol"*
>   *HybridCar.fuelType() -> ఏది call అవ్వాలి? -> Ambiguity!*

```
      Vehicle
     /       \
ElectricCar  PetrolCar
     \       /
     HybridCar   <-- Diamond shape!
```

JavaScript లో Class Level లో Problem లేదు

```
// JavaScript class లో multiple extend కుదరదు:
// class HybridCar extends ElectricCar, PetrolCar { } // SyntaxError

// Java లో కూడా multiple CLASS inheritance కుదరదు (compile error)
// కానీ Java లో ఒక class అనేక INTERFACES implement చేయవచ్చు – అలా diamond ని manage
చేస్తుంది
// JavaScript classes కి single inheritance మాత్రమే (interfaces official గా లేవు)
```

Mixin తో Diamond Problem Solve చేయడం

```
const ElectricCarMixin = (Base) =>
  class extends Base {
    fuelType() {
      console.log("Electricity తో నడుస్తుంది");
    }
  };

const PetrolCarMixin = (Base) =>
  class extends Base {
    fuelType() {
      console.log("Petrol తో నడుస్తుంది");
    }
  };

class Vehicle {
  move() {
    console.log("Vehicle కదులుతోంది...");
  }
}

// HybridCar – రెండు behaviors combine
class HybridCar extends ElectricCarMixin(PetrolCarMixin(Vehicle)) {
  // Override చేసి ambiguity resolve చేయడం
  fuelType() {
    console.log("Hybrid – రెండూ వాడగలదు!");
    // Specific versions call చేయడం (optional)
    ElectricCarMixin(class {}).prototype.fuelType.call(this);
    PetrolCarMixin(class {}).prototype.fuelType.call(this);
```

```
  }
}

const hybrid = new HybridCar();
hybrid.fuelType();
// Hybrid - రెండూ వాడగలదు!
// Electricity తో నడుస్తుంది
// Petrol తో నడుస్తుంది
```

---

16. Method Chaining

వివరణ

Method Chaining అంటే ఒక్కొక్క method వరుసగా ఒకే line లో call చేయడం.
దీనికి ప్రతి method `this` return చేయాలి.

Real-life Scenario

> *Builder pattern: Pizza తయారు చేయేటప్పుడు*
> *`pizza.addCheese().addTomatoes().addPeppers().bake()`*

Code

```
class QueryBuilder {
  #table = "";
  #conditions = [];
  #selectedColumns = ["*"];
  #limitValue = null;
  #orderByColumn = null;

  from(table) {
    this.#table = table;
    return this; // Method chaining కోసం this return చేయడం
  }

  select(...columns) {
    this.#selectedColumns = columns;
    return this;
  }

  where(condition) {
    this.#conditions.push(condition);
    return this;
  }

  limit(n) {
    this.#limitValue = n;
    return this;
  }
```

```
  orderBy(column) {
    this.#orderByColumn = column;
    return this;
  }

  build() {
    let query = `SELECT ${this.#selectedColumns.join(", ")} FROM ${this.#table}`;
    if (this.#conditions.length > 0) {
      query += ` WHERE ${this.#conditions.join(" AND ")}`;
    }
    if (this.#orderByColumn) {
      query += ` ORDER BY ${this.#orderByColumn}`;
    }
    if (this.#limitValue) {
      query += ` LIMIT ${this.#limitValue}`;
    }
    return query;
  }
}

// Method chaining వాడి query build చేయడం
const query = new QueryBuilder()
  .from("students")
  .select("name", "grade", "age")
  .where("age > 18")
  .where("grade = 'A'")
  .orderBy("name")
  .limit(10)
  .build();

console.log(query);
// SELECT name, grade, age FROM students WHERE age > 18 AND grade = 'A' ORDER BY
name LIMIT 10
```

---

17. Composition vs Inheritance

వివరణ

- **Inheritance**: "is-a" relationship -> Dog **is a** Animal
- **Composition**: "has-a" relationship -> Car **has a** Engine

Real-life Scenario

> ***Inheritance**: Manager **is a** Employee **is a** Person*
> *****Composition**: Car **has an** Engine, **has** Wheels, **has** AC*

Code - Composition

```
// Composition - parts తో object తయారు చేయడం
class Engine {
  constructor(horsepower) {
```

```
    this.horsepower = horsepower;
  }

  start() {
    console.log(`Engine (${this.horsepower}HP) start అయింది!`);
  }

  stop() {
    console.log("Engine stop అయింది.");
  }
}

class GPS {
  navigate(destination) {
    console.log(`${destination} కి navigate చేస్తున్నాం...`);
  }
}

class AirConditioner {
  on() {
    console.log("AC on చేయబడింది!");
  }
  off() {
    console.log("AC off చేయబడింది.");
  }
}

// Car has-a Engine, has-a GPS, has-a AirConditioner
class Car {
  constructor(model, horsepower) {
    this.model = model;
    // Composition - objects as properties
    this.engine = new Engine(horsepower);
    this.gps = new GPS();
    this.ac = new AirConditioner();
  }

  startJourney(destination) {
    console.log(`\n${this.model} journey మొదలు:`);
    this.engine.start();
    this.ac.on();
    this.gps.navigate(destination);
  }

  endJourney() {
    console.log(`\n${this.model} journey ముగింపు:`);
    this.ac.off();
    this.engine.stop();
  }
}

const car = new Car("Tesla Model 3", 450);
```

```
car.startJourney("Hyderabad");
// Tesla Model 3 journey మొదలు:
// Engine (450HP) start అయింది!
// AC on చేయబడింది!
// Hyderabad కి navigate చేస్తున్నాం...

car.endJourney();
// Tesla Model 3 journey ముగింపు:
// AC off చేయబడింది.
// Engine stop అయింది.
```

Inheritance vs Composition - ఎప్పుడు ఏది వాడాలి?

Situation	వాడాల్సినది
"is-a" relationship (Dog is Animal)	**Inheritance**
"has-a" relationship (Car has Engine)	**Composition**
Simple hierarchy	**Inheritance**
Flexible, loosely coupled code	**Composition**
Multiple sources from behaviour	**Composition / Mixins**

> ***Rule of thumb:*** *"Favour composition over inheritance" - complex hierarchies వద్దు.*

---

18. Complete Real-world Example - School Management System

అన్ని OOP concepts ఒకే example లో చూద్దాం!

```
// ==========================================
// Real World: School Management System
// అన్ని OOP concepts ఒకే example లో!
// ==========================================

// 1. Base Abstract Class - Abstraction
class Person {
  #name; // Encapsulation - private field
  #age; // Encapsulation - private field

  constructor(name, age) {
    // Abstract class check - directly instantiate చేయకుండా
    if (new.target === Person) {
      throw new Error("Person is abstract - directly create చేయలేరు!");
    }
    this.#name = name;
    this.#age = age;
  }

  // Getters - Encapsulation + Abstraction
  get name() {
```

```
    return this.#name;
  }
  get age() {
    return this.#age;
  }

  // Abstract method - subclass తప్పనిసరిగా implement చేయాలి
  getRole() {
    throw new Error("getRole() implement చేయాలి!");
  }

  // Concrete method - common behaviour
  introduce() {
    // getRole() - Polymorphism! Different output for Teacher vs Student
    console.log(
      `నా పేరు ${this.#name}, వయసు ${this.#age}. నేను ${this.getRole()}.`,
    );
  }
}

// ------------------------------------------------
// 2. Teacher class - Inheritance
// ------------------------------------------------
class Teacher extends Person {
  #subject; // Encapsulation
  #salary; // Encapsulation

  constructor(name, age, subject, salary) {
    super(name, age); // Constructor Chaining - Person constructor call
    this.#subject = subject;
    this.#salary = salary;
  }

  get subject() {
    return this.#subject;
  }

  // Method Overriding - getRole() ని Teacher version తో override
  getRole() {
    return `${this.#subject} Teacher`;
  }

  teach() {
    console.log(`${this.name} ${this.#subject} class చెప్తున్నారు!`);
  }

  // Static method - object లేకుండా call చేయవచ్చు
  static getMinSalary() {
    return 30000;
  }
}
```

```
// ------------------------------------------------
// 3. Student class - Inheritance
// ------------------------------------------------
class Student extends Person {
  #grade;
  #rollNo;
  static #totalStudents = 0; // Static property - shared across all instances

  constructor(name, age, rollNo, grade) {
    super(name, age); // Constructor Chaining - Person constructor call
    this.#rollNo = rollNo;
    this.#grade = grade;
    Student.#totalStudents++;
  }

  get grade() {
    return this.#grade;
  }

  // Setter with validation - Encapsulation
  set grade(g) {
    const valid = ["A", "B", "C", "D", "F"];
    if (!valid.includes(g)) {
      console.log(`"${g}" invalid grade! Valid: A, B, C, D, F only.`);
      return;
    }
    this.#grade = g;
    console.log(`Grade updated to: ${g}`);
  }

  // Method Overriding - getRole() ని Student version తో override
  getRole() {
    return `Student (Roll No: ${this.#rollNo})`;
  }

  study() {
    console.log(`${this.name} చదువుతున్నారు!`);
  }

  static getTotalStudents() {
    return Student.#totalStudents;
  }
}

// ==========================================
// Testing all concepts together!
// ==========================================

console.log("=== School Management System ===\n");

// Object creation
const teacher = new Teacher("Ravi Sir", 40, "Mathematics", 75000);
```

```javascript
const s1 = new Student("Yaswanth", 22, 101, "A");
const s2 = new Student("Chaitanya", 21, 102, "B");
const s3 = new Student("Priya", 20, 103, "A");

// Polymorphism - same introduce() method, different outputs
console.log("--- Introductions (Polymorphism) ---");
teacher.introduce(); // నా పేరు Ravi Sir... నేను Mathematics Teacher.
s1.introduce(); // నా పేరు Yaswanth... నేను Student (Roll No: 101).
s2.introduce(); // నా పేరు Chaitanya... నేను Student (Roll No: 102).

// Specific methods
console.log("\n--- Activities ---");
teacher.teach(); // Ravi Sir Mathematics class చెప్తున్నారు!
s1.study(); // Yaswanth చదువుతున్నారు!

// Encapsulation - Setter validation
console.log("\n--- Grade Update (Encapsulation + Setter) ---");
s1.grade = "X"; // "X" invalid grade! Valid: A, B, C, D, F only.
s1.grade = "A+"; // "A+" invalid grade!
s1.grade = "B"; // Grade updated to: B

console.log(`Yaswanth's current grade: ${s1.grade}`); // B

// Static
console.log("\n--- Static Methods ---");
console.log(`Total Students: ${Student.getTotalStudents()}`); // 3
console.log(`Min Teacher Salary: Rs.${Teacher.getMinSalary()}`); // Rs.30000

// Polymorphism - loop లో process చేయడం
console.log("\n--- School Directory (Runtime Polymorphism) ---");
const people = [teacher, s1, s2, s3];
people.forEach((person) => {
  person.introduce(); // Each person introduces themselves differently!
});

// instanceof - inheritance chain check
console.log("\n--- instanceof checks ---");
console.log(s1 instanceof Student); // true
console.log(s1 instanceof Person); // true - inheritance chain!
console.log(teacher instanceof Teacher); // true
console.log(teacher instanceof Person); // true

// Abstract class cannot be instantiated
try {
  const p = new Person("Test", 25); // Error!
} catch (e) {
  console.log(`\nAbstraction working: ${e.message}`);
  // Person is abstract - directly create చేయలేరు!
}
```

---

19. Memory Tips Table - మర్చిపోకూడదంటే

Concept	గుర్తుంచుకోవడానికి Trick
Class	Blueprint - factory mold లాంటిది
Object	Blueprint నుండి తయారైన actual item
Constructor	Object పుట్టినప్పుడు automatically call అయ్యే function
this	"నా" - ఆ specific object ని refer చేస్తుంది
Encapsulation	ATM machine - లోపలి circuits దాచి buttons మాత్రమే ఇస్తుంది
#private	# తో మొదలయ్యే fields బయట access కుదరవు
Getter	Property లా చదవడం, కానీ method లా పని చేస్తుంది
Setter	Property లా set చేయడం + automatic validation
Abstraction	TV remote - complex circuit దాచి simple buttons
Inheritance	తండ్రి నుండి కొడుకుకి వచ్చే వారసత్వం
extends	Child class parent ని extend చేయడం
super()	Parent class constructor ని call చేయడం
Method Overriding	Child class లో parent method ని redefine చేయడం
Polymorphism	pay() - credit card, UPI, cash వేర్వేరుగా behave చేస్తాయి
Static	Object అక్కర్లేదు - class పేరుతో నేరుగా call చేయవచ్చు
Prototype Chain	Method వెతికే chain - object > parent > grandparent > null
Mixin	Multiple sources నుండి behaviour borrow చేయడం
Composition	has-a: Car has Engine (parts తో object)
Inheritance	is-a: Dog is Animal (parent-child hierarchy)

Quick Syntax Reference

```
// Complete Class Syntax
class MyClass extends ParentClass {
  #privateField; // Private field
  static staticProp = "shared value"; // Static property

  constructor(params) {
    super(params); // Parent constructor MUST be called first
    this.publicProp = params; // Public property
    this.#privateField = "val"; // Private property
  }
```

```
  get myGetter() {
    // Getter
    return this.#privateField;
  }

  set mySetter(val) {
    // Setter
    this.#privateField = val;
  }

  instanceMethod() {
    // Regular method
    return this; // Return this for method chaining
  }

  static classMethod() {} // Static method – no object needed

  toString() {
    // Override Object's default method
    return `MyClass(${this.publicProp})`;
  }
}

// Usage
const obj = new MyClass("test"); // Object creation
obj.instanceMethod().instanceMethod(); // Method chaining
MyClass.classMethod(); // Static method call
console.log(obj instanceof MyClass); // instanceof check – true
console.log(obj instanceof ParentClass); // Inheritance chain check – true
```

> ***గుర్తుంచుకో:*** *OOP అంటే real world ని code లో model చేయడం.*
> *ప్రతి concept కి real-life analogy ఆలోచించు - అప్పుడు మర్చిపోలేవు!*

---

20. Private Methods

వివరణ

Private Fields (`#field`) గురించి చెప్పాం. అదే విధంగా **Private Methods** కూడా ఉంటాయి!
`#methodName()` - class బయట ఈ method call చేయలేరు.
Internal helper logic దాచడానికి ఉపయోగపడుతుంది.

Real-life Scenario

> *Bank లో లోపల password hashing, validation జరుగుతుంది - కానీ user ఆ process directly call చేయలేరు.*
> *అదే private methods!*

Code

```javascript
class BankAccount {
  #balance;
  #pin;

  constructor(initialBalance, pin) {
    this.#balance = initialBalance;
    this.#pin = pin;
  }

  // Private method - class బయట call చేయలేరు
  #validatePin(enteredPin) {
    return enteredPin === this.#pin;
  }

  // Private method - internal use మాత్రమే
  #logTransaction(type, amount) {
    console.log(`[LOG] ${type}: Rs.${amount} | Balance: Rs.${this.#balance}`);
  }

  // Public method - private methods ని internally వాడుతోంది
  withdraw(amount, enteredPin) {
    // Private method call - internally మాత్రమే possible
    if (!this.#validatePin(enteredPin)) {
      console.log("Wrong PIN! Access denied.");
      return;
    }

    if (amount > this.#balance) {
      console.log("Insufficient balance!");
      return;
    }

    this.#balance -= amount;
    this.#logTransaction("WITHDRAWAL", amount); // Private method internally call
    console.log(`Rs.${amount} withdrawn successfully.`);
  }

  getBalance(enteredPin) {
    if (!this.#validatePin(enteredPin)) {
      console.log("Wrong PIN!");
      return null;
    }
    return this.#balance;
  }
}

const acc = new BankAccount(10000, 1234);

acc.withdraw(2000, 1234); // [LOG] WITHDRAWAL: Rs.2000 | Balance: Rs.8000
acc.withdraw(500, 9999); // Wrong PIN! Access denied.
```

```
console.log(acc.getBalance(1234)); // 8000

// Private method directly call చేయడానికి ప్రయత్నిస్తే
// acc.#validatePin(1234); // SyntaxError: Private field '#validatePin'
// acc.#logTransaction("TEST", 100); // SyntaxError!
```

Private Fields vs Private Methods Summary

	Private Field	Private Method
Syntax	`#fieldName`	`#methodName()`
Purpose	Private data store	Internal logic helper
Access	Only inside class	Only inside class
Example	`#balance`	`#validatePin()`

---

21. Custom Error Classes (Error Inheritance)

వివరణ

JavaScript లో built-in `Error` class extend చేసి **custom errors** తయారు చేయవచ్చు.
ఇది OOP Inheritance యొక్క real-world application.

Real-life Scenario

> *General "Error" కాకుండా specific errors:*
>
> - *ValidationError: Input wrong అయినప్పుడు*
> - *NetworkError: API call fail అయినప్పుడు*
> - *AuthenticationError: Login fail అయినప్పుడు*

Code

```
// Custom Error Classes – Inheritance వాడడం
class AppError extends Error {
  constructor(message, errorCode) {
    super(message); // Error class constructor call
    this.name = this.constructor.name; // Error name automatically set
    this.errorCode = errorCode;
    this.timestamp = new Date().toISOString();
  }
}

class ValidationError extends AppError {
  constructor(field, message) {
    super(message, "VALIDATION_ERROR");
    this.field = field; // Which field failed validation
  }
}
```

```javascript
class NetworkError extends AppError {
  constructor(url, statusCode) {
    super(`Network request failed: ${url}`, "NETWORK_ERROR");
    this.url = url;
    this.statusCode = statusCode;
  }
}

class AuthenticationError extends AppError {
  constructor(reason) {
    super(`Authentication failed: ${reason}`, "AUTH_ERROR");
  }
}

// Custom errors వాడే function
function login(username, password) {
  if (!username || username.length < 3) {
    throw new ValidationError(
      "username",
      "Username minimum 3 characters ఉండాలి!",
    );
  }

  if (!password || password.length < 8) {
    throw new ValidationError(
      "password",
      "Password minimum 8 characters ఉండాలి!",
    );
  }

  if (username === "admin" && password !== "admin123456") {
    throw new AuthenticationError("Wrong password");
  }

  console.log(`${username} successfully logged in!`);
}

// Error types ని specifically catch చేయవచ్చు
function processLogin(username, password) {
  try {
    login(username, password);
  } catch (error) {
    if (error instanceof ValidationError) {
      console.log(
        `Validation Error on field "${error.field}": ${error.message}`,
      );
    } else if (error instanceof AuthenticationError) {
      console.log(`Auth Error: ${error.message}`);
    } else if (error instanceof NetworkError) {
      console.log(`Network Error (${error.statusCode}): ${error.message}`);
    } else {
```

```
      console.log(`Unknown Error: ${error.message}`);
    }
  }
}

processLogin("ab", "pass"); // Validation Error on field "username": ...
processLogin("admin", "wrongpass"); // Auth Error: Authentication failed...
processLogin("admin", "admin123456"); // admin successfully logged in!

// instanceof తో specific error type check
try {
  throw new ValidationError("email", "Invalid email format");
} catch (e) {
  console.log(e instanceof ValidationError); // true
  console.log(e instanceof AppError); // true - inheritance chain!
  console.log(e instanceof Error); // true - Error class inherit!
  console.log(e.name); // ValidationError
  console.log(e.errorCode); // VALIDATION_ERROR
  console.log(e.field); // email
}
```

---

22. toString() మరియు valueOf() Override

వివరణ

JavaScript లో objects ని string లా లేదా number లా use చేసేటప్పుడు, JavaScript automatically `toString()` లేదా `valueOf()` call చేస్తుంది.
వీటిని override చేయడం OOP లో important concept.

Real-life Scenario

> *Money object ని `console.log` చేసినప్పుడు "Rs. 5000" లా చూపించాలి కానీ `{ amount: 5000 }` లా కాదు.*

Code

```
class Money {
  constructor(amount, currency = "INR") {
    this.amount = amount;
    this.currency = currency;
  }

  // toString() - string context లో auto-called
  toString() {
    return `${this.currency} ${this.amount.toFixed(2)}`;
  }

  // valueOf() - number/math context లో auto-called
  valueOf() {
    return this.amount;
```

```
  }

  // Symbol.toPrimitive - modern way (overrides toString/valueOf)
  [Symbol.toPrimitive](hint) {
    if (hint === "string") {
      return `${this.currency} ${this.amount.toFixed(2)}`;
    }
    if (hint === "number") {
      return this.amount;
    }
    return this.amount; // default
  }
}

const price = new Money(1500.5);
const tax = new Money(270);

// String context -> Symbol.toPrimitive "string" hint (toString/valueOf ని override
చేస్తుంది)
console.log(`Price: ${price}`); // Price: INR 1500.50   (template literal ->
"string" hint)

// జాగ్రత్త! "+" operator "string" hint కాదు, "default" hint వాడుతుంది.
// మన Symbol.toPrimitive లో default -> this.amount (number). అందుకే string కాదు,
number వస్తుంది:
console.log("Price is: " + price); // Price is: 1500.5     (NOT "INR 1500.50"!)

// Number/default context -> Symbol.toPrimitive "number"/"default" hint
console.log(price + tax); // 1770.5 (numbers add!)
console.log(price > 1000); // true
console.log(price * 2); // 3001

// Explicit conversion
console.log(String(price)); // INR 1500.50   ("string" hint)
console.log(Number(price)); // 1500.5        ("number" hint)
```

> ***Hint precedence గుర్తుంచుకో:***
>
> - *`Symbol.toPrimitive` ఉంటే అదే గెలుస్తుంది - `toString()` / `valueOf()` ని పూర్తిగా override చేస్తుంది.*
> - *Template literal, `String()` -> **"string"** hint*
> - *Math operators (`-` , `*` , `>`), `Number()` -> **"number"** hint*
> - *`+` operator, `==` -> **"default"** hint (అందుకే `"..." + obj` string కాకపోవచ్చు!)*

Date class example

```
class CustomDate {
  constructor(year, month, day) {
    this.year = year;
    this.month = month;
    this.day = day;
```

```
  }

  toString() {
    const m = String(this.month).padStart(2, "0");
    const d = String(this.day).padStart(2, "0");
    return `${this.year}-${m}-${d}`;
  }

  valueOf() {
    return new Date(this.year, this.month - 1, this.day).getTime();
  }
}

const d1 = new CustomDate(2024, 1, 15);
const d2 = new CustomDate(2024, 6, 30);

console.log(`Date: ${d1}`); // Date: 2024-01-15
console.log(d1 < d2); // true (valueOf() comparison)
console.log(d2 - d1); // milliseconds difference (number subtraction)
```

---

23. Object.create() - Prototype-based Object Creation

వివరణ

`new ClassName()` కాకుండా, `Object.create()` వాడి prototype directly set చేసి object తయారు చేయవచ్చు. ఇది JavaScript యొక్క low-level inheritance mechanism.

Code

```
// Prototype object
const animalProto = {
  eat() {
    console.log(`${this.name} తింటోంది.`);
  },

  sleep() {
    console.log(`${this.name} నిద్రపోతోంది.`);
  },

  toString() {
    return `Animal(${this.name})`;
  },
};

// Object.create() - animalProto ని prototype గా set చేసి object create చేయడం
const dog = Object.create(animalProto);
dog.name = "Tommy";
dog.breed = "Labrador";
dog.bark = function () {
  console.log(`${this.name} భౌ అంటోంది!`);
```

```
};

dog.eat(); // Tommy తింటోంది. (prototype method)
dog.sleep(); // Tommy నిద్రపోతోంది. (prototype method)
dog.bark(); // Tommy భౌ అంటోంది! (own method)

// Prototype chain verify
console.log(Object.getPrototypeOf(dog) === animalProto); // true
console.log(dog.hasOwnProperty("name")); // true (own property)
console.log(dog.hasOwnProperty("eat")); // false (prototype method)

// null prototype - no inheritance at all
const pureObj = Object.create(null);
pureObj.key = "value";
// pureObj.toString(); // Error! No Object.prototype methods!
```

---

24. Factory Functions - Class కి Alternative

వివరణ

`class` syntax వాడకుండా, **Factory Functions** వాడి objects తయారు చేయవచ్చు.
`new` keyword అక్కర్లేదు - function call చేస్తే object return అవుతుంది.

Real-life Scenario

> *Ice cream factory - ఏ flavor అడిగినా same structure తో ice cream return చేస్తుంది.*

Code

```
// Factory Function - class కి alternative
function createPerson(name, age) {
  // Private variable (closure తో)
  let _secretCode = Math.random().toString(36).slice(2);

  return {
    name,
    age,

    greet() {
      console.log(`హలో! నా పేరు ${name}, వయసు ${age}.`);
    },

    getSecretCode() {
      return _secretCode; // Closure - private access
    },

    updateAge(newAge) {
      age = newAge; // Closure variable update
      console.log(`Age updated to ${newAge}`);
    },
  };
```

```
}

const person1 = createPerson("Yaswanth", 22);
const person2 = createPerson("Chaitanya", 25);

person1.greet(); // హాలో! నా పేరు Yaswanth, వయసు 22.
person2.greet(); // హాలో! నా పేరు Chaitanya, వయసు 25.

console.log(person1.getSecretCode()); // Random code (private via closure)
person1.updateAge(23); // Age updated to 23

// Factory Function vs Class – comparison
// Factory Function: new keyword లేదు, prototype chain లేదు, closure privacy
// Class: new keyword, prototype chain, # private fields
```

Factory Function Advantages

```
// Factory function with inheritance simulation
function createAnimal(name, sound) {
  return {
    name,
    makeSound() {
      console.log(`${name} ${sound}!`);
    },
    sleep() {
      console.log(`${name} నిద్రపోతోంది.`);
    },
  };
}

function createDog(name) {
  const animal = createAnimal(name, "భౌ భౌ");
  return {
    ...animal, // Spread – parent properties include చేయడం
    fetch() {
      console.log(`${name} ball తీసుకొస్తోంది!`);
    },
  };
}

const dog = createDog("Tommy");
dog.makeSound(); // Tommy భౌ భౌ!
dog.fetch(); // Tommy ball తీసుకొస్తోంది!
dog.sleep(); // Tommy నిద్రపోతోంది.
```

---

25. Singleton Pattern

వివరణ

Singleton అంటే ఒక class నుండి **ఒకే ఒక instance** మాత్రమే create అవ్వాలి అనే design pattern. Global state manage చేయడానికి వాడతాం.

Real-life Scenario

> *Application లో **database connection** ఒక్కటే ఉండాలి - ప్రతిసారి new connection తెరవకూడదు.*
> *****Logger** ఒక్కటే ఉండాలి - అన్ని places నుండి same logger వాడాలి.*

Code

```
class DatabaseConnection {
  static #instance = null; // Private static - single instance store

  #connection;
  #queryCount = 0;

  // Private constructor - directly `new` చేయలేరు (simulation)
  constructor(host, port) {
    if (DatabaseConnection.#instance) {
      throw new Error(
        "DatabaseConnection is a Singleton! getInstance() వాడండి.",
      );
    }
    this.#connection = { host, port, connected: true };
    console.log(`Database connected to ${host}:${port}`);
  }

  // Static factory method - single instance return చేయడం
  static getInstance(host = "localhost", port = 5432) {
    if (!DatabaseConnection.#instance) {
      DatabaseConnection.#instance = new DatabaseConnection(host, port);
    }
    return DatabaseConnection.#instance;
  }

  query(sql) {
    this.#queryCount++;
    console.log(`Query #${this.#queryCount}: ${sql}`);
    return `Results for: ${sql}`;
  }

  getQueryCount() {
    return this.#queryCount;
  }

  static resetInstance() {
    DatabaseConnection.#instance = null; // Testing కోసం
  }
}

// First call - connection create అవుతుంది
```

```
const db1 = DatabaseConnection.getInstance("db.server.com", 5432);
// Database connected to db.server.com:5432

// Second call - same instance return అవుతుంది (no new connection!)
const db2 = DatabaseConnection.getInstance();

console.log(db1 === db2); // true - same object!

db1.query("SELECT * FROM users"); // Query #1: SELECT * FROM users
db2.query("SELECT * FROM products"); // Query #2: SELECT * FROM products

console.log(db1.getQueryCount()); // 2 (both queries counted on SAME instance)

// Direct new - error!
try {
  const db3 = new DatabaseConnection("other.server.com", 5432);
} catch (e) {
  console.log(e.message); // DatabaseConnection is a Singleton!
}
```

---

26. Object Immutability - Object.freeze() మరియు Object.seal()

వివరణ

OOP లో sometimes objects ని **immutable** (మార్పులేనివి) చేయాలి.
JavaScript లో దీనికి `Object.freeze()` మరియు `Object.seal()` వాడతాం.

Real-life Scenario

> *Application settings object - ఒకసారి load అయిన తర్వాత మరెవ్వరూ మార్చకూడదు.*
> *Constants object - values fixed గా ఉండాలి.*

Code

```
class Config {
  constructor(settings) {
    Object.assign(this, settings);
    Object.freeze(this); // Object freeze - ఇక మార్పులు impossible!
  }
}

const appConfig = new Config({
  appName: "MyApp",
  version: "1.0.0",
  maxRetries: 3,
  apiUrl: "https://api.myapp.com",
});

console.log(appConfig.appName); // MyApp

// Frozen object ని మార్చడానికి / add చేయడానికి ప్రయత్నిస్తే:
```

```
//   Strict mode (ES modules, class methods, TS) లో -> TypeError throw అవుతుంది
//   Non-strict (sloppy) mode లో                    -> silently ignore అవుతుంది
// రెండు modes లోనూ safe గా ఉండాలంటే try/catch వాడతాం:
try {
  appConfig.appName = "HackedApp"; // modify - blocked
} catch (e) {
  console.log("Frozen! appName మార్చలేకపోయాం.");
}
try {
  appConfig.newProp = "secret"; // add - blocked
} catch (e) {
  console.log("Frozen! కొత్త property add చేయలేకపోయాం.");
}

console.log(appConfig.appName); // Still "MyApp" - unchanged!
console.log(appConfig.newProp); // undefined - add కాలేదు

// Freeze check
console.log(Object.isFrozen(appConfig)); // true
```

Object.freeze() vs Object.seal() తేడా

```
const frozenObj = Object.freeze({ a: 1, b: 2 });
// Cannot: add, remove, or modify properties

const sealedObj = Object.seal({ a: 1, b: 2 });
// Cannot: add or remove properties
// CAN: modify existing property values

sealedObj.a = 999; // Allowed! (existing property modify - ఎప్పుడూ works)

// Add / delete: strict mode -> TypeError, sloppy mode -> silently ignored
try {
  sealedObj.c = 3; // new property add - not allowed
} catch (e) {
  console.log("Sealed! కొత్త property add చేయలేకపోయాం.");
}
try {
  delete sealedObj.b; // delete - not allowed
} catch (e) {
  console.log("Sealed! property delete చేయలేకపోయాం.");
}

console.log(sealedObj); // { a: 999, b: 2 }
```

	`Object.freeze()`	`Object.seal()`
Add new properties	No	No
Delete properties	No	No

Modify values	No	Yes
Use case	True immutability	Shape-only locked

⚠️ జాగ్రత్త: Object.freeze() SHALLOW (deep కాదు!)

```
// Object.freeze() ఒక్క level మాత్రమే freeze చేస్తుంది – nested objects mutable గానే ఉంటాయి!
const config = Object.freeze({ name: "App", settings: { debug: true } });
config.settings.debug = false; // లోపలి object freeze కాలేదు – మారిపోతుంది!
console.log(config.settings.debug); // false (మారింది!)

// Deep freeze – recursive గా అన్ని levels freeze చేయాలి
function deepFreeze(obj) {
  Object.keys(obj).forEach((key) => {
    if (typeof obj[key] === "object" && obj[key] !== null) deepFreeze(obj[key]);
  });
  return Object.freeze(obj);
}
const safe = deepFreeze({ name: "App", settings: { debug: true } });
try {
  safe.settings.debug = false;
} catch (e) {}
console.log(safe.settings.debug); // true (deep freeze కాపాడింది)
```

Enum Simulation - Object.freeze తో Constants

```
// JavaScript లో official enum లేదు – Object.freeze తో simulate చేస్తాం
const Status = Object.freeze({
  ACTIVE: "active",
  INACTIVE: "inactive",
  BANNED: "banned",
});

console.log(Status.ACTIVE); // active
// Status.ACTIVE = "hacked";  // frozen – మారదు (accidental change నుండి రక్షణ)

function checkUser(status) {
  if (status === Status.BANNED) return "🚫 Access denied";
  return "✅ Welcome";
}
console.log(checkUser(Status.BANNED)); // 🚫 Access denied
```

---

27. Symbol.iterator - Objects ని Iterable చేయడం

వివరణ

OOP లో custom objects ని `for...of` loop తో iterate చేయాలంటే, `Symbol.iterator` method implement చేయాలి.

ఇది JavaScript objects ని iterable చేసే mechanism.

Real-life Scenario

> *Shopping cart object లో `for...of` తో items loop చేయడం.*

Code

```
class ShoppingCart {
  #items = [];

  addItem(name, price, qty = 1) {
    this.#items.push({ name, price, qty });
    return this; // Method chaining
  }

  removeItem(name) {
    this.#items = this.#items.filter((item) => item.name !== name);
    return this;
  }

  get total() {
    return this.#items.reduce((sum, item) => sum + item.price * item.qty, 0);
  }

  get count() {
    return this.#items.length;
  }

  // Symbol.iterator - for...of loop enable చేయడం
  [Symbol.iterator]() {
    let index = 0;
    const items = this.#items;

    return {
      next() {
        if (index < items.length) {
          return { value: items[index++], done: false };
        }
        return { value: undefined, done: true };
      },
    };
  }

  toString() {
    return `ShoppingCart(${this.#items.length} items, Total: Rs.${this.total})`;
  }
}

const cart = new ShoppingCart();
```

```
cart.addItem("Rice", 60, 2).addItem("Dal", 120, 1).addItem("Oil", 180, 1);

// Symbol.iterator వల్ల for...of possible!
for (const item of cart) {
  console.log(
    `${item.name}: Rs.${item.price} x ${item.qty} = Rs.${item.price * item.qty}`,
  );
}
// Rice: Rs.60 x 2 = Rs.120
// Dal: Rs.120 x 1 = Rs.120
// Oil: Rs.180 x 1 = Rs.180

console.log(String(cart)); // ShoppingCart(3 items, Total: Rs.420)
console.log(cart.total); // 420

// Spread operator కూడా works!
const itemArray = [...cart];
console.log(itemArray.length); // 3

// Destructuring కూడా works!
const [firstItem, ...rest] = cart;
console.log(firstItem.name); // Rice
```

---

28. Duck Typing - Interface-like Behaviour

వివరణ

JavaScript లో official interfaces లేవు.
కానీ **Duck Typing** concept వాడతాం:

> *"If it walks like a duck and quacks like a duck, it is a duck."*
> *అంటే - object కి ఆ method ఉంటే, అది ఆ type లా treat చేస్తాం.*

Real-life Scenario

> *Printable interface: ఏ object కైనా `print()` method ఉంటే, అది printable అని treat చేయవచ్చు.*

Code

```
// "Interface" simulate - runtime check
function assertImplements(obj, interfaceName, methods) {
  for (const method of methods) {
    if (typeof obj[method] !== "function") {
      throw new Error(
        `${obj.constructor.name} must implement "${method}()" from ${interfaceName}
interface`,
      );
    }
  }
}
```

```
// Required methods list (like interface definition)
const DRAWABLE_INTERFACE = ["draw", "resize", "getArea"];
const SAVEABLE_INTERFACE = ["save", "load"];

class Circle {
  constructor(radius) {
    this.radius = radius;
    assertImplements(this, "Drawable", DRAWABLE_INTERFACE); // Runtime check
  }

  draw() {
    console.log(`Circle (r=${this.radius}) drawing...`);
  }
  resize(factor) {
    this.radius *= factor;
  }
  getArea() {
    return Math.PI * this.radius ** 2;
  }
}

class Rectangle {
  constructor(w, h) {
    this.width = w;
    this.height = h;
    assertImplements(this, "Drawable", DRAWABLE_INTERFACE);
  }

  draw() {
    console.log(`Rectangle (${this.width}x${this.height}) drawing...`);
  }
  resize(factor) {
    this.width *= factor;
    this.height *= factor;
  }
  getArea() {
    return this.width * this.height;
  }
}

// Duck typing - draw() ఉంటే, అది drawable!
function renderAll(shapes) {
  shapes.forEach((shape) => {
    shape.draw();
    console.log(`  Area: ${shape.getArea().toFixed(2)}`);
  });
}

const shapes = [new Circle(5), new Rectangle(4, 6), new Circle(3)];

renderAll(shapes);
// Circle (r=5) drawing...
```

```
//   Area: 78.54
// Rectangle (4x6) drawing...
//   Area: 24.00
// Circle (r=3) drawing...
//   Area: 28.27
```

---

29. Protected Simulation (Public/Private/Protected)

వివరణ

Java లో 3 access modifiers ఉంటాయి:

- **public** - అందరికీ accessible
- **protected** - same class + child classes కి accessible
- **private** - same class మాత్రమే

JavaScript లో officially `protected` లేదు. కానీ naming convention తో simulate చేయవచ్చు.

Code

```
class Animal {
  // Public - అందరికీ accessible
  name;

  // Protected simulation - convention: single underscore prefix
  // Child classes వాడవచ్చు, external code వాడకూడదు (by convention)
  _energy;
  _age;

  // Private - class మాత్రమే
  #secretId;

  constructor(name, age) {
    this.name = name; // public
    this._energy = 100; // protected (by convention)
    this._age = age; // protected (by convention)
    this.#secretId = Math.random().toString(36).slice(2); // private
  }

  // Public method
  eat(food) {
    this._energy += 10; // protected field - subclass logic shared
    console.log(`${this.name} ${food} తింటోంది. Energy: ${this._energy}`);
  }

  // Protected method (by convention - single underscore)
  _breathe() {
    this._energy -= 1;
    return "breathing...";
  }
```

```
  // Private method
  #generateId() {
    return this.#secretId;
  }

  // Public - uses private internally
  getInfo() {
    return `${this.name} (ID: ${this.#generateId()}, Age: ${this._age})`;
  }
}

class Dog extends Animal {
  #breed;

  constructor(name, age, breed) {
    super(name, age);
    this.#breed = breed;
  }

  bark() {
    // Protected field access - child class can use _energy
    this._energy -= 5; // Energy వాడడం
    console.log(`${this.name} భౌ అంటోంది! Energy: ${this._energy}`);
  }

  run() {
    // Protected method access from child class
    this._breathe(); // OK - child class can use _breathe()
    this._energy -= 15;
    console.log(`${this.name} పరిగెత్తింది! Energy: ${this._energy}`);
  }

  getBreed() {
    return this.#breed;
  }
}

const dog = new Dog("Tommy", 3, "Labrador");
dog.eat("Bone"); // Tommy Bone తింటోంది. Energy: 110
dog.bark(); // Tommy భౌ అంటోంది! Energy: 105
dog.run(); // Tommy పరిగెత్తింది! Energy: 89
console.log(dog.getInfo()); // Tommy (ID: xyz123, Age: 3)

// External code - convention బట్టి _protected వాడకూడదు
// dog._energy = 999; // Technically possible but BAD practice (underscore = "don't
touch")
```

Access Levels Summary Table

Level	JavaScript	Convention	Example

Public	Default	No prefix	`this.name`
Protected	Convention only	`_prefix`	`this._energy`
Private	# syntax	`#prefix`	`this.#balance`

---

30. Operator Overloading Simulation

వివరణ

Java, C++ లో operator overloading ఉంది (`+` , `-` , `==` override చేయడం).
JavaScript లో direct operator overloading లేదు, కానీ `valueOf()` మరియు `Symbol.toPrimitive` వాడి simulate చేయవచ్చు.

Code

```
class Vector2D {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  // + operator simulation
  add(other) {
    return new Vector2D(this.x + other.x, this.y + other.y);
  }

  // - operator simulation
  subtract(other) {
    return new Vector2D(this.x - other.x, this.y - other.y);
  }

  // Scalar multiplication
  multiply(scalar) {
    return new Vector2D(this.x * scalar, this.y * scalar);
  }

  // Magnitude (length)
  get magnitude() {
    return Math.sqrt(this.x ** 2 + this.y ** 2);
  }

  // Dot product
  dot(other) {
    return this.x * other.x + this.y * other.y;
  }

  // Equality check
  equals(other) {
    return this.x === other.x && this.y === other.y;
  }
```

```
  // toString - string context లో auto-called
  toString() {
    return `Vector2D(${this.x}, ${this.y})`;
  }

  // valueOf - numeric context
  valueOf() {
    return this.magnitude;
  }
}

const v1 = new Vector2D(3, 4);
const v2 = new Vector2D(1, 2);

console.log(v1.add(v2).toString()); // Vector2D(4, 6)
console.log(v1.subtract(v2).toString()); // Vector2D(2, 2)
console.log(v1.multiply(2).toString()); // Vector2D(6, 8)
console.log(v1.magnitude); // 5
console.log(v1.dot(v2)); // 11

// valueOf() - numeric comparison possible!
console.log(v1 > v2); // true (5 > 2.236)
console.log(`${v1}`); // Vector2D(3, 4)
```

---

31. bind(), call(), apply() - this ని Explicit గా Bind చేయడం

వివరణ

Topic 4 లో arrow function తో `this` problem solve చేశాం. కానీ classic గా, JavaScript లో ప్రతి function కి `bind()` , `call()` , `apply()` అనే 3 methods ఉంటాయి - వీటితో `this` ఏ object ని refer చేయాలో మనమే **explicit గా** చెప్పవచ్చు.

Method	ఏం చేస్తుంది	Arguments	వెంటనే call అవుతుందా?
`call()`	this మార్చి వెంటనే call	comma తో separate	అవును
`apply()`	this మార్చి వెంటనే call	ఒక **array** గా	అవును
`bind()`	this fix చేసి **కొత్త function** return	comma తో separate	కాదు (తర్వాత call చేయాలి)

> ***గుర్తుంచుకో:*** *call = **C**omma, apply = **A**rray, bind = **B**ound copy (కొత్త function).*

Real-life Scenario

> *Button కి `handleClick` method ని event listener కి pass చేసినప్పుడు `this` పోతుంది. `.bind(this)` తో `this` ని permanent గా attach చేస్తే, ఎక్కడ call చేసినా సరైన object ని చూపిస్తుంది.*

Code

```
class Button {
  constructor(label) {
    this.label = label;
    this.clicks = 0;
  }

  handleClick() {
    this.clicks++;
    console.log(`${this.label} button ${this.clicks} times click అయింది`);
  }
}

const saveBtn = new Button("Save");

// PROBLEM: method ని reference గా వేరే చోటికి pass చేస్తే this పోతుంది
const looseHandler = saveBtn.handleClick;
try {
  looseHandler(); // this = undefined -> error
} catch (e) {
  console.log(`Error: ${e.message}`); // Cannot read properties of undefined
}

// SOLUTION: bind() - this ని permanent గా fix చేసి కొత్త function return చేస్తుంది
const boundHandler = saveBtn.handleClick.bind(saveBtn);
boundHandler(); // Save button 1 times click అయింది
boundHandler(); // Save button 2 times click అయింది
```

call() మరియు apply() - Method Borrowing

```
class Dog {
  constructor(name) {
    this.name = name;
  }

  describe(mood, activity) {
    console.log(`${this.name} ${mood}గా ఉంది, ${activity} చేస్తోంది.`);
  }
}

const dog = new Dog("Tommy");

// Cat object కి describe() method లేదు - కానీ Dog ది borrow చేయవచ్చు!
const cat = { name: "Kitty" };

// call() - this ని మార్చి వెంటనే call (arguments comma తో separate)
dog.describe.call(cat, "happy", "ఆడుకోవడం");
// Kitty happyగా ఉంది, ఆడుకోవడం చేస్తోంది.

// apply() - same పని కానీ arguments ఒక array గా pass చేయడం
```

```
dog.describe.apply(cat, ["sleepy", "నిద్రపోవడం"]);
// Kitty sleepyగా ఉంది, నిద్రపోవడం చేస్తోంది.
```

bind() తో Partial Application

```
function multiply(a, b) {
  return a * b;
}

// a = 2 ని ముందే fix చేసి కొత్త function తయారు చేయడం
const double = multiply.bind(null, 2);
console.log(double(5)); // 10  (2 * 5)
console.log(double(10)); // 20  (2 * 10)
```

Key Points

- `call` / `apply` వెంటనే execute చేస్తాయి; `bind` ఒక కొత్త function ఇస్తుంది (తర్వాత call చేయాలి)
- Method borrowing - ఒక class method ని వేరే object మీద నడపవచ్చు
- Event handlers, callbacks లో `this` పోకుండా `bind` లేదా arrow function వాడతాం
- Arrow function auto-bind చేస్తుంది; `bind` manual control ఇస్తుంది

---

32. SOLID Principles

వివరణ

SOLID అంటే మంచి, maintainable OOP code రాయడానికి 5 design principles. ఇవి Robert C. Martin (Uncle Bob) చెప్పినవి. వీటిని follow చేస్తే code flexible, testable, మరియు easy-to-change గా ఉంటుంది.

అక్షరం	Principle	సూత్రం (ఒక్క వాక్యం)
S	Single Responsibility	ఒక class కి ఒకే ఒక పని (మారడానికి ఒకే కారణం)
O	Open/Closed	Extension కి open, modification కి closed
L	Liskov Substitution	Parent స్థానంలో child పెట్టినా code పని చేయాలి
I	Interface Segregation	పెద్ద interface కంటే చిన్న, specific interfaces మంచివి
D	Dependency Inversion	Concrete class మీద కాదు, abstraction మీద depend అవ్వాలి

S - Single Responsibility Principle

> *ఒక class database work, email work, business logic అన్నీ చేస్తే - ఏదో ఒకటి మారితే మొత్తం class touch చేయాలి. బదులుగా ఒక్కో పని ఒక్కో class కి ఇస్తాం.*

```
// WRONG: User class database + email అన్నీ చేస్తోంది
// CORRECT: బాధ్యతలు విడదీయడం
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
```

```
  }
}

class UserRepository {
  // ఒకే బాధ్యత: save/load
  save(user) {
    console.log(`${user.name} ని database లో save చేశాం.`);
  }
}

class EmailService {
  // ఒకే బాధ్యత: email పంపడం
  sendWelcome(user) {
    console.log(`${user.email} కి welcome email పంపాం.`);
  }
}

const u = new User("Yaswanth", "y@mail.com");
new UserRepository().save(u); // Yaswanth ని database లో save చేశాం.
new EmailService().sendWelcome(u); // y@mail.com కి welcome email పంపాం.
```

O - Open/Closed Principle

> *కొత్త shape add చేయాలంటే existing code (Shape, totalArea) మార్చకూడదు - కొత్త class రాస్తే చాలు.*

```
class Shape {
  area() {
    throw new Error("area() implement చేయాలి!");
  }
}

class Circle extends Shape {
  constructor(r) {
    super();
    this.r = r;
  }
  area() {
    return Math.PI * this.r ** 2;
  }
}

class Square extends Shape {
  constructor(s) {
    super();
    this.s = s;
  }
  area() {
    return this.s ** 2;
  }
}
```

```
// కొత్త shape — పాత code ఏమీ మార్చకుండా add చేయవచ్చు (Open for extension)
class Triangle extends Shape {
  constructor(b, h) {
    super();
    this.b = b;
    this.h = h;
  }
  area() {
    return 0.5 * this.b * this.h;
  }
}

function totalArea(shapes) {
  return shapes.reduce((sum, s) => sum + s.area(), 0);
}

console.log(
  totalArea([new Circle(2), new Square(3), new Triangle(4, 5)]).toFixed(2),
);
// 31.57
```

L - Liskov Substitution Principle

> *Parent (Bird) ని ఆశించే చోట ఏ child (Sparrow, Penguin) పెట్టినా program break అవ్వకూడదు.*

```
class Bird {
  move() {
    console.log("Bird కదులుతోంది.");
  }
}

class Sparrow extends Bird {
  move() {
    console.log("Sparrow ఎగురుతోంది.");
  }
}

class Penguin extends Bird {
  move() {
    console.log("Penguin నడుస్తోంది.");
  } // ఎగరదు, కానీ move() contract ని honour చేస్తుంది
}

function makeItMove(bird) {
  bird.move(); // ఏ Bird అయినా substitute చేయవచ్చు
}

makeItMove(new Sparrow()); // Sparrow ఎగురుతోంది.
makeItMove(new Penguin()); // Penguin నడుస్తోంది.
```

I - Interface Segregation Principle

> *Robot కి `eat()` అవసరం లేదు. ఒక పెద్ద interface లో అన్నీ కూరకుండా, చిన్న capabilities (Workable, Eatable) విడదీసి కావలసినవి మాత్రమే combine చేస్తాం (mixins వాడి - Topic 14).*

```
const Workable = (Base) =>
  class extends Base {
    work() {
      console.log(`${this.name} పని చేస్తోంది.`);
    }
  };
const Eatable = (Base) =>
  class extends Base {
    eat() {
      console.log(`${this.name} తింటోంది.`);
    }
  };

class Entity {
  constructor(name) {
    this.name = name;
  }
}

class Human extends Workable(Eatable(Entity)) {} // work + eat రెండూ
class Robot extends Workable(Entity) {} // work మాత్రమే (eat బలవంతం కాదు)

new Human("Ravi").work(); // Ravi పని చేస్తోంది.
new Robot("R2D2").work(); // R2D2 పని చేస్తోంది.
```

D - Dependency Inversion Principle

> *`Notification` ఏ specific channel (SMS/Email) మీద కాకుండా, ఒక `send()` contract మీద depend అవుతుంది. Channel ని బయటి నుండి inject చేస్తాం (dependency injection - Topic 17 composition లాంటిదే).*

```
class SMSChannel {
  send(msg) {
    console.log(`SMS: ${msg}`);
  }
}
class EmailChannel {
  send(msg) {
    console.log(`Email: ${msg}`);
  }
}

class Notification {
  constructor(channel) {
    // dependency ని inject చేయడం
    this.channel = channel;
  }
```

```
  notify(msg) {
    this.channel.send(msg);
  }
}

new Notification(new SMSChannel()).notify("Hello via SMS"); // SMS: Hello via SMS
new Notification(new EmailChannel()).notify("Hello via Email"); // Email: Hello via
Email
```

Key Points

- SOLID = maintainable, testable, flexible OOP code కి guidelines
- ఈ document లో నేర్చుకున్న inheritance, polymorphism, mixins, composition - అన్నీ ఇక్కడ కలిసి పని చేస్తాయి
- Real projects లో ఈ principles code quality ని కాపాడతాయి

---

33. Modern Class Features (ES2022+)

వివరణ

JavaScript classes కి ఇటీవల (ES2022) కొన్ని powerful features వచ్చాయి:

- **Static initialization blocks** (`static { }`) - complex static setup కోసం
- **Private brand checks** (`#field in obj`) - ఒక object ఈ class ది అవునా అని safe గా check చేయడం
- `Symbol.hasInstance` - `instanceof` behaviour ని customize చేయడం

Static Initialization Block

> *Static property ని ఒక్క line లో కాకుండా, కొన్ని steps తో setup చేయాలంటే* `static { }` *block వాడతాం. Class load అయినప్పుడు ఒక్కసారే run అవుతుంది.*

```
class Config {
  static settings = {};
  static environment;

  // Static block - complex logic తో static values setup
  static {
    const isProd = false; // ఉదా: environment నుండి వస్తుంది అనుకోండి
    Config.environment = isProd ? "production" : "development";
    Config.settings = {
      timeout: isProd ? 30 : 5,
      debug: !isProd,
    };
    console.log(`Config loaded for: ${Config.environment}`);
  }
}

console.log(Config.environment); // development
console.log(Config.settings.timeout); // 5
console.log(Config.settings.debug); // true
```

Private Brand Check (`#field in obj`)

> `instanceof` ని fool చేయవచ్చు, కానీ private field మాత్రం ఆ class లోపల create అయిన object లోనే ఉంటుంది. అందుకే `#field in obj` అనేది object ఏ class ది అని safe గా చెప్పే "brand check".

```
class Account {
  #balance = 0;

  static isAccount(obj) {
    // #balance ఆ object లో ఉందా? — private field brand check.
    // జాగ్రత్త: primitive/null కి `in` operator error ఇస్తుంది,
    // అందుకే ముందు object అని confirm చేస్తాం.
    return typeof obj === "object" && obj !== null && #balance in obj;
  }

  deposit(amt) {
    this.#balance += amt;
    return this;
  }
  getBalance() {
    return this.#balance;
  }
}

const realAccount = new Account();
const fakeAccount = { balance: 100 }; // Account లా కనిపిస్తుంది కానీ కాదు

console.log(Account.isAccount(realAccount)); // true
console.log(Account.isAccount(fakeAccount)); // false  (#balance లేదు)
console.log(Account.isAccount("hello")); // false  (primitive — error రాదు!)
```

Symbol.hasInstance - instanceof ని Customize చేయడం

> `Symbol.hasInstance` override చేస్తే, `instanceof` ఏం చెప్పాలో మనమే నిర్ణయించవచ్చు. ఇక్కడ `area()` method ఉన్న ఏ object అయినా "Shape" గా treat అవుతుంది (duck typing - Topic 28 లాంటిది).

```
class Shape {
  static [Symbol.hasInstance](obj) {
    return obj != null && typeof obj.area === "function";
  }
}

const circle = {
  area() {
    return 3.14;
  },
}; // Shape extend చేయలేదు!

console.log(circle instanceof Shape); // true  (area() ఉంది కాబట్టి)
console.log({} instanceof Shape); // false (area() లేదు)
```

Key Points

- `static { }` - class definition లోనే ఒక్కసారి run అయ్యే setup block
- `#field in obj` - private field brand check (try/catch అవసరం లేదు, కానీ primitive కి type guard పెట్టాలి)
- `Symbol.hasInstance` - `instanceof` operator ని మన rules తో నడపడం

---

34. Async in OOP (async methods, Promises, for await)

వివరణ

నిజ ప్రపంచ classes లో methods తరచూ **async** - data fetch, DB call, file read. JavaScript లో `async` / `await` తో వీటిని synchronous లా చదవగలిగేలా రాస్తాం. Class methods కూడా `async` కావచ్చు.

Real-life Scenario

> ***Restaurant order***: *order ఇచ్చాక వెంటనే food రాదు - కొంత సమయం wait చేయాలి (Promise). ఈలోపు నువ్వు వేరే పని చేయవచ్చు. Food ready అయ్యాక తింటావు (* `await` *).*

Code

```
class UserService {
  // async method – ఎప్పుడూ Promise return చేస్తుంది
  async fetchUser(id) {
    // await – Promise పూర్తయ్యేదాకా ఆగుతుంది (కానీ thread block కాదు)
    await new Promise((r) => setTimeout(r, 5)); // API delay ని simulate
    if (id <= 0) throw new Error("Invalid id");
    return { id, name: "Yaswanth" };
  }

  // Error handling – async లో try/catch
  async safeFetch(id) {
    try {
      const user = await this.fetchUser(id);
      console.log(`✅ దొరికింది: ${user.name}`);
      return user;
    } catch (e) {
      console.log(`❌ Error: ${e.message}`);
      return null;
    }
  }

  // Async iterator – stream data ని for await తో loop చేయడం
  async *streamOrders() {
    for (let i = 1; i <= 3; i++) {
      await new Promise((r) => setTimeout(r, 5));
      yield `Order #${i}`;
    }
  }
}
```

```
// Top-level await (ES modules లో) లేదా async function లోపల
const service = new UserService();
const user = await service.fetchUser(101);
console.log(user); // { id: 101, name: 'Yaswanth' }
await service.safeFetch(-1); // ❌ Error: Invalid id

// for await...of - async iterator ని consume చేయడం
for await (const order of service.streamOrders()) {
  console.log("📦 " + order); // 📦 Order #1 ... #2 ... #3
}
```

Key Points

- `async` method ఎప్పుడూ **Promise** return చేస్తుంది; `await` దాన్ని విప్పుతుంది
- Async code లో errors ని `try/catch` తో పట్టుకోవాలి (లేదా `.catch()`)
- `async *` + `for await...of` = async iterator (streams, paginated APIs)
- `await` thread ని block చేయదు - event loop మిగతా పని కొనసాగిస్తుంది
- ⚠️ Constructor `async` కాలేదు - async setup కి static factory (`static async create()`) వాడు

---

35. Generators మరియు Iterators (function*, yield)

వివరణ

Generator = అవసరమైనప్పుడు (lazily), ఒక్కో value ని `yield` చేసే special function (`function*`). Topic 27 లో iterator ని manual గా (`next()` , `{value, done}`) రాశాం - generator అదే పని **చాలా సులభంగా** చేస్తుంది.

Real-life Scenario

> ***Netflix "next episode"**: అన్ని episodes ఒకేసారి download చేయవు. నువ్వు అడిగినప్పుడు (`next`) తర్వాతిది ఇస్తుంది. Generator అలా on-demand values ఇస్తుంది - infinite sequences కూడా possible.*

Code

```
// function* + yield - iterator ని automatic గా తయారు చేస్తుంది
class NumberRange {
  constructor(start, end, step = 1) {
    this.start = start;
    this.end = end;
    this.step = step;
  }
  // Topic 27 లోని పొడవైన next() code బదులు - ఒక్క generator!
  *[Symbol.iterator]() {
    for (let i = this.start; i <= this.end; i += this.step) {
      yield i; // ఒక్కో value ని ఇక్కడ "pause చేసి" ఇస్తుంది
    }
  }
}

console.log([...new NumberRange(1, 10, 2)]); // [1, 3, 5, 7, 9]
for (const n of new NumberRange(1, 3)) console.log(n); // 1, 2, 3
```

```
// Infinite generator – lazy కాబట్టి memory పేలదు
function* idGenerator() {
  let id = 1;
  while (true) yield id++; // ఆగకుండా, కానీ అడిగినప్పుడే
}
const gen = idGenerator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3

// yield* – మరో iterable కి delegate చేయడం
function* combined() {
  yield* [1, 2]; // array నుండి
  yield* "ab"; // string నుండి
}
console.log([...combined()]); // [1, 2, 'a', 'b']
```

Manual Iterator (Topic 27) vs Generator

	Manual `[Symbol.iterator]()`	Generator `*[Symbol.iterator]()`
Code	`next()`, index, `{value, done}`	`yield` - చాలా తక్కువ code
State	మనమే track చేయాలి	Automatic (pause/resume)
Infinite	కష్టం	సులభం (`while(true) yield`)

> ***గుర్తుంచుకో:*** *`function*` = generator function. `yield` = "ఇక్కడ ఆగి, ఈ value ఇవ్వు, తర్వాత ఇక్కడి నుండి కొనసాగు."*

---

36. Class Fields మరియు Arrow Methods (Auto-bind)

వివరణ

Modern JavaScript లో constructor లేకుండానే **class fields** declare చేయవచ్చు (public, private `#`, static). ముఖ్యంగా - **arrow function field** గా method రాస్తే, `this` **automatic గా bind** అవుతుంది (Topic 4, 31 లోని `this` problem కి అందమైన పరిష్కారం).

Real-life Scenario

> *Topic 31 లో event handler కి `this` పోకుండా `.bind(this)` వాడాం. Arrow method field (`handleClick = () => {}`) అదే auto-bind ని constructor లో bind రాయకుండానే ఇస్తుంది - React components లో అత్యంత common.*

Code

```
class Counter {
  count = 0; // public field – constructor అవసరం లేదు
  #history = []; // private field
  static instances = 0; // static field (అన్ని objects కి shared)
```

```
  constructor() {
    Counter.instances++;
  }

  // Arrow method field - "this" ఎప్పుడూ ఈ object కే bound (auto-bind!)
  increment = () => {
    this.count++;
    this.#history.push(this.count);
  };

  // Regular method - reference గా pass చేస్తే "this" పోతుంది
  reset() {
    this.count = 0;
  }

  getHistory() {
    return [...this.#history];
  }
}

const c = new Counter();

// Method ని reference గా తీసుకున్నా this పోదు (arrow auto-bind వల్ల)
const inc = c.increment;
inc();
inc();
console.log(c.count); // 2  (this సరిగ్గా pointing)
console.log(c.getHistory()); // [1, 2]
console.log(Counter.instances); // 1

// Regular method reference గా తీస్తే this పోతుంది (comparison)
const badReset = c.reset;
try {
  badReset();
} catch (e) {
  console.log("Regular method this పోయింది!");
}
```

bind() vs Arrow Method Field

	`this.m = this.m.bind(this)`	`m = () => {}` field
ఎక్కడ	Constructor లో manual	Field declaration లో
Boilerplate	ఎక్కువ	తక్కువ (auto)
React handlers	పాత style	ఆధునిక style

> ***గమనిక:*** *Arrow method field ప్రతి instance కి ఒక కొత్త function తయారు చేస్తుంది (prototype మీద కాదు). చాలా objects + memory-critical అయితే regular method +* `bind` *మంచిది.*

---

37. Symbols, new.target మరియు Reflection

వివరణ

ఈ document లో `Symbol.iterator` (27), `Symbol.toPrimitive` (22), `Symbol.hasInstance` (33), `new.target` (10) వాడాం - కానీ అవి ఏమిటో విడిగా చెప్పలేదు. ఇక్కడ ఆ పునాది concepts.

Symbol - Unique Keys

> ***Symbol*** *= ప్రతిదీ 100% unique. రెండు symbols description ఒకటే అయినా సమానం కావు. Object లో collision-free, quasi-private keys కి వాడతాం.*

```
const id1 = Symbol("id");
const id2 = Symbol("id");
console.log(id1 === id2); // false (unique!)

const user = { name: "Yaswanth", [id1]: 101 };
console.log(user[id1]); // 101
console.log(Object.keys(user)); // ['name'] — symbol keys hidden!
```

Well-known Symbols = JavaScript behaviour ని hook చేసే built-in symbols: `Symbol.iterator` (for...of), `Symbol.toPrimitive` (type coercion), `Symbol.hasInstance` (instanceof), `Symbol.asyncIterator` (for await).

new.target - "new తో పిలిచారా?"

> *`new.target` = constructor ని ఏ class తో `new` చేశారో చెప్తుంది. Abstract class guard కి (Topic 10) వాడాం.*

```
class Base {
  constructor() {
    console.log("new.target:", new.target.name);
  }
}
class Derived extends Base {}
new Base(); // new.target: Base
new Derived(); // new.target: Derived (subclass పేరు!)
```

Reflection - Object ని Inspect చేయడం

```
const p = { a: 1, b: 2 };
console.log(Object.keys(p)); // ['a', 'b']
console.log(Object.entries(p)); // [['a', 1], ['b', 2]]

// Object.defineProperty — property మీద fine control (descriptor)
const obj = {};
Object.defineProperty(obj, "PI", {
  value: 3.14,
  writable: false,
  enumerable: false,
});
```

```
console.log(obj.PI); // 3.14
console.log(Object.keys(obj)); // [] (enumerable:false → hidden)
```

WeakMap - `#` రాకముందు Privacy Pattern

> `#private` *fields రాకముందు, private data ని* ***WeakMap*** *లో దాచేవారు. Object garbage collect అయితే దాని data కూడా auto-remove (memory leak లేదు).*

```
const _balance = new WeakMap(); // private store (object బయట)
class Account {
  constructor(b) {
    _balance.set(this, b);
  }
  getBalance() {
    return _balance.get(this);
  }
}
const acc = new Account(500);
console.log(acc.getBalance()); // 500
console.log(Object.keys(acc)); // [] - balance ఎక్కడా కనిపించదు
```

Key Points

- **Symbol** = unique key; well-known symbols language behaviour ని customize చేస్తాయి
- **new.target** = ఏ class `new` అయిందో (abstract guards కి)
- **Reflection** = `Object.keys/entries`, `defineProperty` తో objects ని inspect/control
- **WeakMap** = `#` కి ప్రత్యామ్నాయ (memory-safe) privacy; metadata attach చేయడానికి

అన్నీ Missing Topics Summary - Updated Table of Contents

ఇప్పుడు document లో cover అయినవి:

#	Topic	Status
1	OOP అంటే ఏమిటి?	Covered
2	Class మరియు Object	Covered
3	Constructor	Covered
4	this keyword	Covered
5	Encapsulation (Private Fields)	Covered
6	Getters మరియు Setters	Covered
7	Inheritance	Covered
8	Constructor Chaining (super)	Covered
9	Method Overriding	Covered
10	Abstraction	Covered

11	Polymorphism	Covered
12	Static Methods & Properties	Covered
13	Prototype Chain	Covered
14	Mixins	Covered
15	Diamond Problem	Covered
16	Method Chaining	Covered
17	Composition vs Inheritance	Covered
18	Complete Real-world Example	Covered
19	Memory Tips Table	Covered
20	**Private Methods**	Added
21	**Custom Error Classes**	Added
22	**toString() / valueOf() Override**	Added
23	**Object.create()**	Added
24	**Factory Functions**	Added
25	**Singleton Pattern**	Added
26	**Object.freeze() / seal()**	Added
27	**Symbol.iterator (Iterable)**	Added
28	**Duck Typing**	Added
29	**Protected Simulation**	Added
30	**Operator Overloading Simulation**	Added
31	**bind() / call() / apply()**	Added
32	**SOLID Principles**	Added
33	**Modern Class Features (ES2022+)**	Added
34	**Async in OOP (async/await, for await)**	Added
35	_Generators (function, yield)_*	Added
36	**Class Fields & Arrow Methods (auto-bind)**	Added
37	**Symbols, new.target & Reflection**	Added

JavaScript OOP - Complete Telugu Guide (Updated with all missing topics)